



Security Architecture & Threat-Model Whitepaper

A supervised, hardware-rooted, human-gated AI agent fleet for Google Cloud — its design, the formal threat model it was subjected to before release, the findings that review produced, and the verified remediation and hardened control set that resulted.

OPEN SOURCE

github.com/paracoding-ai/paracoding-oss

Scott McDonald

July 13, 2026

Technical Reference · Version 1.0 · Prepared for security-architecture review
Methodology: STRIDE-per-element | NIST SP 800-30 risk register | MITRE ATT&CK mapping | trust-boundary decomposition

Contents

1	Executive summary
2	Introduction, scope & audience
3	System architecture
4	Threat-modeling methodology
5	Trust-boundary decomposition
6	Assets & threat actors
7	Findings of the security review (F1–F13)
8	Attack-tree analysis
9	Remediation & verification
10	The hardened control set
11	Residual & accepted risk
12	Control mapping (NIST SP 800-53)
13	Honest posture & claims
14	Conclusion
A	Appendix A — full risk register
B	Appendix B — ATT&CK technique coverage
C	Appendix C — methodology & references

How to read this document

This whitepaper is deliberately candid. It does not present the system as flawless; it presents a system that was **formally threat-modeled, found to have specific weaknesses, and hardened before release** — and it shows the evidence for each fix. Every control is labelled **[DEFAULT]** (present in all deployments), **[RECOMMENDED]** (shipped as code, expected for production), or **[OPTIONAL]** (documented hardening for an organization context). A security posture you cannot verify is not a security posture.

1 Executive summary

Paracoding provisions a small fleet of role-scoped AI agents — from a git repository into a fresh Google Cloud project — that perform real infrastructure and operations work under continuous human supervision. It was engineered against one non-negotiable principle: **an autonomous agent must never be able to take a consequential action unattended**. An agent proposes; a human commits.

Before the reference implementation was released, it was subjected to a formal threat model using STRIDE-per-element decomposition, a NIST SP 800-30 risk register, MITRE ATT&CK technique mapping, and explicit trust-boundary and attack-tree analysis. That review produced thirteen findings (F1–F13), including one rated critical: an unattended execution path that could reach root privilege without a human in the loop. Rather than ship and patch later, the critical and high-severity findings were remediated first, and — under a “prove, don’t claim” discipline — each remediation was verified by re-running the original attack path and demonstrating it was blocked.

The headline is not “no vulnerabilities were found.” It is “we found them, fixed them, and proved the fix — before release.”

The resulting posture rests on eight pillars: a hardware root of trust anchored in Google Titan silicon; Secure Boot and boot-integrity attestation; per-agent cryptographic identity that cannot be spoofed; keyless, least-privilege cloud authentication; a human-apply gate with no unattended path to root; an end-to-end audit trail; full reproducibility from source; and a defense-in-depth network edge built on Google Cloud Armor. This document describes each, distinguishes what every deployment enforces from what is recommended for production, and states the residual risks plainly.

Summary of outcomes

Findings identified	13 (1 critical, 5 high, 4 medium, 3 low), pre-release
Critical / high remediated	All, before release, each verified by re-running the attack path
Core safety property	No unattended path to root; every privileged action human-committed
Verification discipline	Prove-don’t-claim: a fix is “done” only when the exploit is demonstrably blocked
Reproducibility	Whole fleet rebuilds from source; disaster-recovery drill measured in seconds-to-minutes
Honest boundaries	Single-operator blast radius and recommended-vs-default edge disclosed, not hidden

2 Introduction, scope & audience

2.1 Purpose

This is a technical security document written for an engineer or reviewer deciding whether Paracoding is safe to run in a real environment — a lab, an internal platform team, or an organization's cloud estate. It describes concrete mechanisms, the boundaries between components, the threat model applied to them, and the verified controls that resulted. It is not a brochure, and it deliberately avoids language it cannot substantiate.

2.2 Scope

- **In scope:** the agent control plane, the privileged execution path and its human gate, agent identity and authorization, the cloud and network trust boundaries, secret handling, the boot and platform integrity layer, and reproducibility / disaster recovery.
- **Out of scope:** the internal reasoning of the language models themselves; the security of the operator's endpoint device; and any organization-specific compliance program, which is the adopter's responsibility. Where a control interacts with these, it is noted.

2.3 A note on candor

The most important editorial decision in this document is to include the findings of the security review rather than only the finished controls. A whitepaper that lists strengths without ever admitting what was hard is not credible to a security reader. Each finding below is presented *with* its remediation and the evidence that the remediation holds. That pairing — finding, fix, proof — is the difference between a demonstration of engineering rigor and an attack playbook; this document is firmly the former.

2.4 Labeling convention

Controls carry one of three labels. **[DEFAULT]** means the control is present in every deployment, including a minimal one. **[RECOMMENDED]** means the control ships as reproducible code and is the expected production posture, but is not part of a bare minimal deploy. **[OPTIONAL]** means the control is documented hardening that depends on an organizational context (for example, an organization policy) and is enabled by the operator. This convention is used consistently so that no control is silently overclaimed.

3 System architecture

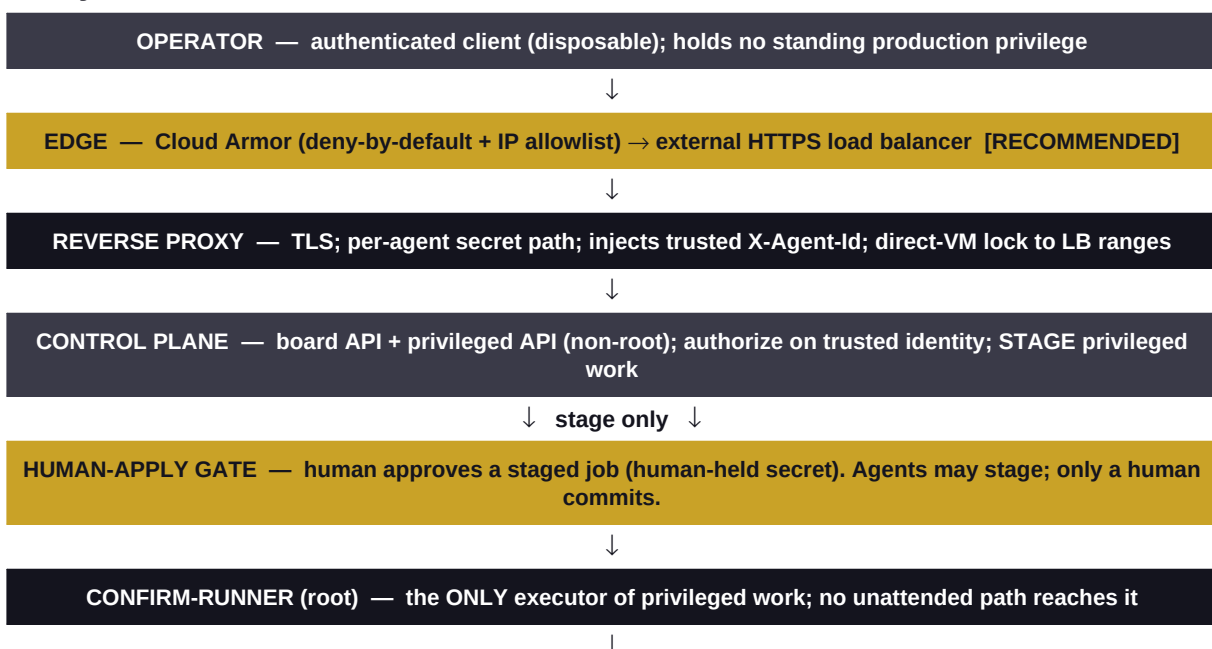
The components, what each is trusted to do, and how privilege is deliberately partitioned.

Paracoding separates *reasoning* from *privilege*. The agents (and the human's client) reason and propose; a small set of tightly-scoped server-side components hold the actual capability to change anything, and the most dangerous of those — execution as root — is reachable only through an explicit human approval. The durable identity and memory of each agent live server-side, which means the client is disposable: losing or replacing a client never grants or leaks privilege.

3.1 Component inventory

Reverse proxy (front door)	service	Terminate TLS; bind each agent's secret path to a trusted identity; serve public sites separately from the control plane
Control plane — board API	service (non-root)	Hold work items, journal, agent state; authorize tools on the proxy-asserted identity
Control plane — privileged API	service (non-root)	Accept privileged job requests behind a capability gate; stage — never execute inline
Confirm-runner	root	Execute a staged privileged job only after a human approval backed by a human-held secret
Worker sandbox (browser / model tasks)	isolated non-root	Run untrusted web content and generation tasks with a deny-by-default domain policy
State store	local, service-owned	Persist board and journal; never exposed directly to the network
Edge: Cloud Armor + load balancer	managed	Deny-by-default source-IP allowlist in front of the control plane [RECOMMENDED]

3.2 Layered view



Design consequence

Because reasoning and privilege are separated, and privilege-to-root is reachable only through a human approval, a failure of the reasoning layer — a confused, misled, or prompt-injected agent — cannot by itself change production. The blast radius of “the AI did something wrong” is bounded by the human gate, by construction.

4 Threat-modeling methodology

Why the review used four complementary lenses, and what each contributed.

A single technique tends to miss whole categories of risk. The review deliberately combined four, each covering a blind spot of the others:

STRIDE-per-element

Every component and data flow was examined for Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, and Elevation of privilege. Applying STRIDE *per element* — rather than to the system as a whole — forces each trust boundary to be reasoned about explicitly and surfaces elevation-of-privilege paths that a coarse review glosses over.

Trust-boundary decomposition

The system was drawn as a set of zones separated by boundaries where privilege or trust changes hands. Findings were then anchored to a specific boundary crossing, which makes both the risk and its mitigation concrete rather than abstract.

Attack trees

For the highest-value goals (reach root; read all sensitive data; take over the cloud account), the review built attack trees enumerating the OR/AND paths to each goal. Attack trees are what revealed that several individually-modest weaknesses *chained* into the critical finding.

NIST SP 800-30 risk register + MITRE ATT&CK

Each finding was scored Likelihood × Impact to produce a defensible severity ranking, and the top paths were mapped to MITRE ATT&CK techniques so the analysis speaks the same language as a defender's detection tooling. The register is reproduced in full in Appendix A.

The structural theorem the review produced

The analysis reduced the entire question of root integrity to two properties: (1) the set of jobs an unattended executor may run cannot be extended by an agent, and (2) every job in that set is safe against hostile parameters. Property (1) held. Property (2) did not — two file-handling jobs could be driven with hostile parameters. The remediation did not merely patch those two jobs; it **removed the unattended executor entirely**, so safety no longer depends on property (2) being perfect. Eliminating the category beats hardening every member of it.

5 Trust-boundary decomposition

Six boundaries carry security weight. Each is a point where an attacker who compromised one side must not automatically gain the other. The findings in §7 are anchored to these crossings.

B1	Internet → edge	Only expected origins reach the control plane; public sites stay separate
B2	Edge → reverse proxy	TLS terminated; per-agent secret path required; direct-to-VM bypass closed
B3	Agent → control plane	Authorization on a trusted, proxy-asserted identity — never on a self-claimed one
B4	Control plane → execution	Privileged work is staged, never inline; only the confirm-runner executes it, only after human approval
B5	Fleet → cloud	Short-lived workload credentials only; no long-lived keys cross this line
B6	Untrusted content → agent	Web/model content is processed in an isolated sandbox and cannot itself trigger privileged action

5.1 The boundary that carries the most weight: B4

B4 is where an agent's *intent* becomes an *action*. In the original design a convenience path let an approved-category job cross B4 on a schedule with no human present; the review identified this as the critical weakness (F1) because it collapsed B4 for a whole class of jobs. The remediation re-established B4 as an absolute human gate: today, nothing crosses from “staged” to “executed as root” without a human approval. B4 is now the system's load-bearing control.

6 Assets & threat actors

6.1 Assets, by sensitivity

- **Crown jewels:** root on the fleet host; the cloud account itself; any host-held SSH key; the human-approval secret and the privileged-API capability secret; per-agent connector secrets.
- **Sensitive data:** the board and journal (operational memory); any business content the fleet processes. These are kept out of the public reference by design.
- **Integrity assets:** the job allowlist and job scripts; the infrastructure-as-code that is the source of truth for rebuilds; the reverse-proxy configuration; served websites.
- **Availability assets:** the control-plane endpoint, which is the fleet's lifeline — a careless edge change can lock out every agent, so edge changes are validated before cutover.

6.2 Threat actors considered

TA1	External, unauthenticated	Internet scanning; no secret; the baseline attacker
TA2	Secret-path holder	Obtained one agent connector secret (leak, log, history)
TA3	Prompt-injection	Controls web/document content an agent will process
TA4	Supply-chain	Compromises a dependency or base image
TA5	Compromised operator endpoint	Malware or session theft on the human's device
TA6	Cloud-account takeover	Phish or session theft against the account owner
TA7	Accidental / insider	Operator error; a fat-fingered destructive command

The controls in this document are strongest against TA1–TA4 — the actors reachable through the software itself. TA5 and TA6 concentrate on the human and the account; their *likelihood* is reduced by hardware-backed two-factor authentication and endpoint discipline, but their blast radius in a single-operator deployment is disclosed honestly in §11 rather than papered over.

7 Findings of the security review (F1–F13)

What the review found — each paired with the boundary it crosses and its remediation status. Full risk scoring is in Appendix A.

The review produced thirteen findings. They are summarized here in severity order. Critically, every finding rated medium or above was remediated **before release**; the remediation detail and the evidence appear in §9. This section is the “what we caught” half of the story; §9 is the “and fixed, and proved” half.

F1 [CRITICAL] — Unattended path to root

An hourly executor would run any board item requesting an approved-category job as root, with parameters an agent could set — no human in the loop. This collapsed the execution boundary for a whole class of jobs and was the single most serious finding. (Boundary B4.) [FIXED]

F2 [HIGH] — Credential exfiltration via a file-copy job

A file-synchronization job's source allowlist included a directory holding secrets; driven with hostile parameters through F1, it could copy those secrets into an agent-readable location. (Boundary B4.) [FIXED]

F3 [HIGH] — Self-asserted agent identity

The control plane trusted the agent identity a caller claimed. Any holder of one connector secret could act as any agent and read all board content. (Boundary B3.) [FIXED]

F4 [HIGH] — Prompt-injection to privileged action

Hostile web content processed by an agent could induce it to enqueue a work item that, via F1, reached root — an external-content-to-root chain. (Boundary B6→B4.) [FIXED]

F5 [HIGH] — Cloud-account single point of failure

One cloud account gates billing, compute, source, and inference. Account takeover is total. This is a blast-radius finding: controls reduce its likelihood, not its impact. (Boundary B5.) [FIXED]

F6 [HIGH] — Privileged operator node holds a host key

The operator workstation held an SSH key granting root on the fleet host, making the workstation a privileged node whose compromise is severe. (Boundary B5.) [FIXED]

F7 [MEDIUM] — Public control-plane endpoint

The control plane was reachable on the internet with the per-agent secret path as the only authentication — adequate as a floor, but thin for production exposure. (Boundary B1/B2.) [FIXED]

F8 [MEDIUM] — Sensitive content mirrored off-host

Operational memory containing sensitive content could be mirrored into an external repository's history. (Boundary B5.) [FIXED]

F9 [MEDIUM] — Capability gate bypassed by the unattended path

The privileged-API capability check was bypassed by the F1 route, which reached the runner without presenting the capability. (Boundary B4.) [FIXED]

F10 [MEDIUM] — Deploy job executed an out-of-tree script

A deployment job invoked a script outside the runner's root-owned trust check, so its integrity was assumed rather than verified. (Boundary B4.) [FIXED]

F11 [Low] — Website defacement via a file-write job

File-write jobs targeting the web root could, through F1, alter served content. (*Boundary B4.*) [FIXED]

F12 [Low] — Supply-chain exposure

Dependency and base-image integrity relied on version pinning without full hash pinning. (*Boundary —.*) [FIXED]

F13 [Low] — Data-at-rest granularity

Sensitive fields relied on platform disk encryption without application-level encryption. (*Boundary —.*) [FIXED]

The chain that made F1 critical

F3 (any secret = any identity) + F4 (injected content → a staged request) + F1 (staged request → unattended root) + F2 (a root job that leaks secrets) formed a single path from “an agent reads a malicious web page” to “fleet secrets exfiltrated,” with no human anywhere on it. Severing any link breaks the chain; the remediation severed three.

8 Attack-tree analysis

Two representative trees are shown — as they were analyzed, and as they stand after remediation. The purpose here is not to publish an exploit guide (the specifics are closed); it is to show *how* the reasoning connected findings, and that the connecting links are now cut.

8.1 Tree A — reach root / exfiltrate secrets

Be, or influence, any board-connected agent (via F3 or F4)	Identity is now proxy-bound and server-enforced (F3 closed); injected content cannot reach the privileged path (F4 closed)
Enqueue a request for a file-handling root job	The unattended executor no longer runs file-handling jobs at all
Wait for the unattended executor to run it as root	There is no unattended executor. Execution requires a human approval (F1 closed)
Job copies secrets to an agent-readable location	The secret-bearing source was removed from the job's allowlist (F2 closed)

Verification (see §9): the exact first-and-last steps of this tree were re-executed against the live system. A board item requesting the file-copy job with a source path inside the protected credentials directory was planted; the executor **rejected it as non-allowlisted**, no job was enqueued, and the target directory was never created. The tree is severed at two independent points.

8.2 Tree B — one leaked secret → full compromise

Obtain one connector secret	Still possible in principle; now constrained — see next rows
Use it from any network location	Edge now denies by default and allowlists only expected origins [RECOMMENDED] ; a leaked secret is unusable off-origin
Act as any agent; read all board data	Identity is proxy-bound; the secret maps to one scoped identity, not all (F3 closed)
Pivot to root via the unattended path	No unattended path exists (F1 closed)

Defense-in-depth is visible here: even granting the attacker the initial secret, three independent controls (edge allowlist, bound identity, human gate) must all fail for the goal to be reached. Secret rotation closes the initial step as hygiene.

9 Remediation & verification

The “and fixed, and proved” half. Under a prove-don’t-claim rule, no fix counted as done until its attack path was re-run and shown blocked.

Remediation followed a deliberate sequence: close the critical unattended-root class first, then restore identity, then harden the edge, then capture everything in reproducible code and re-verify by rebuild. Each item below states what changed and the evidence that it holds.

R1 — Eliminate the unattended path to root (closes F1, F2, F9, F10, F11 on that path)

The unattended executor was removed as the route for privileged, file-touching work. Those jobs now require a human approval through the confirm-runner. The fix is by *elimination of the category*, not by patching each job.

Evidence: a planted board item requesting the file-copy job against the protected credentials directory was rejected as non-allowlisted; zero jobs were enqueued; the exfiltration target was never created.

R2 — Bind and enforce agent identity (closes F3)

The reverse proxy now injects a trusted identity header per secret path, set server-side and not settable by the agent; the control plane authorizes on that trusted identity. **Evidence:** a connection asserting a different agent’s identity was rejected — the proxy-bound identity took precedence over the claim, demonstrating that impersonation fails.

R3 — Make the human gate the sole execution route (hardens B4)

Confirm and deny were restricted to a human-only path backed by a human-held secret; an agent connection attempting to approve is rejected. The confirm-runner is the only component that executes staged work.

Evidence: the approval action succeeds only on the human path; the unattended executor count is zero on a live system.

R4 — Harden the network edge (closes F7)

A Google Cloud Armor policy with deny-by-default and a source-IP allowlist was placed in front of the control plane via an external HTTPS load balancer, and the VM’s own web tier was locked to accept only the load-balancer ranges (closing the direct-to-VM bypass). **Evidence:** traffic from outside the allowlisted range received HTTP 403 at the edge; traffic from the allowlisted range completed a full control-plane initialization end-to-end through the load balancer; a direct request to the VM’s own address on the bypass path returned 403.

R5 — Rotate exposed secrets (supports F3/F7)

Connector secrets and the privileged capability secret that had any exposure were rotated; old paths return not-found. **Evidence:** the old path is dead; the new path serves; the acting agent’s own connection was verified still working post-rotation.

R6 — Platform integrity by default (hardware root of trust)

Shielded-VM settings — Titan-rooted vTPM, Secure Boot, integrity monitoring — were baked into the provisioning code so every machine is born with them. **Evidence:** a freshly provisioned host reports Secure Boot enabled and the virtual TPM present and active.

R7 — Capture in code and re-verify by rebuild (closes drift; supports F8/F12)

The edge, the identity injection, and the platform settings were captured in infrastructure-as-code and the whole fleet was rebuilt from source into a clean project. **Evidence:** the rebuilt fleet came up with per-agent identity present, the direct-VM lock returning 403 on the bypass, Secure Boot enabled, services active, and the control

plane responding — i.e., the hardened state is reproducible from source, not a hand-tuned artifact.

Prove, don't claim

The operating rule throughout remediation was that a finding is not “closed” when the code changes — it is closed when the original attack path has been re-run and observed to fail. Every R-item above cites the observation, not the intention. This is the discipline a security reviewer should demand, and it is the one applied here.

10 The hardened control set

This is the resulting posture, organized by pillar, with each control labelled by whether it is default, recommended, or optional.

10.1 Hardware root of trust [DEFAULT]

Every machine is a Google Cloud Shielded VM anchored by Google Titan silicon: a Titan-rooted virtual TPM providing unforgeable device identity and measured boot state; Secure Boot verifying the boot chain against trusted signatures before the OS loads; and integrity monitoring attesting boot measurements against a known-good baseline. The root of trust lives below anything an attacker can alter from inside the OS — verification runs from silicon, up through boot, to the running workload.

10.2 Identity & least privilege

- Per-agent cryptographic identity, proxy-bound and server-enforced [DEFAULT].
- Keyless authentication to Google Cloud via short-lived workload credentials; no long-lived key files exist to leak [DEFAULT].
- Organization-policy enforcement disabling service-account key creation [OPTIONAL] — not set by a bare deploy (a fresh project has no organization), documented as org-context hardening.

10.3 The human-apply gate [DEFAULT] — core control

- Privileged operations are staged, never executed inline.
- A dedicated confirm-runner is the only executor of staged work, and only after explicit human approval backed by a human-held secret.
- Confirm and deny are a human-only path; an agent attempting to approve is rejected.
- No scheduled or automatic route exists from a staged request to root.

10.4 Network edge [RECOMMENDED]

- Cloud Armor policy: deny-by-default with a source-IP allowlist restricting the control plane to expected origins.
- External HTTPS load balancer terminating TLS with the policy attached at the edge.
- Direct-VM lock: the host web tier accepts only load-balancer ranges, closing the bypass.

10.5 Auditability & reproducibility [DEFAULT]

- Audit-trail-first: every action, decision, and approval is journaled durably and reviewably.
- Reproducible-from-source: the hardened fleet rebuilds from its repository; deployed state equals reviewable state; disaster recovery is a repeatable drill.

11 Residual & accepted risk

No honest security document ends at “solved.” The following risks remain and are accepted deliberately, with the rationale stated.

Single-operator blast radius (from F5, F6)

In a small single-operator deployment, the cloud account and the operator endpoint concentrate trust: their compromise is high-impact. Hardware-backed two-factor authentication and endpoint discipline reduce the *likelihood*; they do not reduce the *impact*. This is an accepted trade-off for small teams and is the first thing a multi-operator deployment should distribute (separate break-glass identity, per-role privilege, account-level monitoring).

Edge is recommended, not default

A bare deploy is the reverse proxy plus the secret path. The Cloud Armor edge is shipped as code and is the expected production posture, but a minimal install does not include it. Adopters exposing a control plane to the internet should treat the edge as required, not optional.

Model-layer and content risk

The agents process external content; the sandbox and the human gate contain the consequences, but prompt-injection of an agent’s reasoning is an inherent property of using language models. The mitigation is architectural: keep privileged capability behind the human gate, which this design does.

Supply chain (F12)

Dependency and image integrity rely on version pinning; full hash-pinning and base-image digest pinning are recommended follow-ups and are noted as such rather than claimed as complete.

12 Control mapping (NIST SP 800-53)

A mapping of the control set to control families, provided to aid an adopter's own program. This is a mapping for orientation, not a certification or an attestation of compliance.

AC	Access control	Per-agent scoped identity; least-privilege; human-only approval of privileged actions
IA	Identification & authentication	Proxy-bound, server-enforced agent identity; keyless workload identity to cloud
AU	Audit & accountability	Durable journal of every action, decision, and approval
CM	Configuration management	Infrastructure-as-code; deployed state equals reviewable source; no undocumented drift
SI	System & information integrity	Secure Boot + integrity monitoring; job-integrity trust checks; edge filtering
SC	System & communications protection	TLS at the edge; deny-by-default allowlist; control plane isolated from public sites
CP	Contingency planning	Reproducible-from-source rebuild; disaster-recovery drill
RA	Risk assessment	The formal threat model and risk register in this document

What this table is and is not

It is an orientation aid that maps real mechanisms to the areas an assessor thinks in. It is **not** a claim of compliance, accreditation, or certification against any framework. Paracoding makes no such claim; an adopter's compliance program is the adopter's to run, and this mapping is offered only to make that work easier.

13 Honest posture & claims

13.1 What the system earns

A hardware root of trust; per-agent identity that cannot be spoofed; least-privilege keyless cloud access; a human-approval gate on every privileged action with no unattended path to root; an end-to-end audit trail; and full reproducibility from source. These are default properties of the reference, demonstrated by the evidence in §9, not aspirations.

13.2 What it does not claim

It claims no certification, accreditation, or compliance status it has not been formally audited for. It offers no absolute guarantee of security — no honest system does. It does not describe itself with superlatives it cannot substantiate. Where the industry reaches for words like “certified,” “compliant,” or “unbreakable,” this document deliberately does not.

13.3 Why candor is the point

The value of this architecture is not a badge. It is a posture an engineer can read, verify against the running system, and reason about — including its accepted limits. A reviewer can confirm Secure Boot is on, watch an impersonation attempt fail, observe a bypass request return 403, and rebuild the fleet from source to check that the deployed state matches the code. Trust that can be checked is the only kind worth offering.

14 Conclusion

Paracoding is a supervised AI agent fleet built so that capability is always subordinate to human judgment. It was not shipped on faith: it was formally threat-modeled, its weaknesses were found and ranked, the serious ones were remediated before release, and every remediation was proved by re-running the attack it was meant to stop. The residual risks are stated plainly, and the controls are labelled honestly by whether every deployment enforces them or a production posture adds them.

AI proposes. A human commits. Everything either does is attested, identity-scoped, human-gated, logged, and reproducible from source.

That is the whole thesis, and every section of this document exists to substantiate it with a mechanism and, where it matters most, with evidence.

Paracoding — supervised AI operations you can trust, because you can see, attest, and approve everything they do.

Appendix A Full risk register

NIST SP 800-30 style. L = likelihood, I = impact, rated Low / Med / High; severity weights impact for crown-jewel access. All findings rated medium or above were remediated before release.

F1	Unattended path to root	E	M	H	Crit	Fixed
F2	Credential exfil via file-copy job	I	M	H	High	Fixed
F3	Self-asserted agent identity	S	M	H	High	Fixed
F4	Prompt-injection to privileged action	E	M	H	High	Fixed
F5	Cloud-account single point of failure	E/D	L	H	High	Accepted*
F6	Operator node holds host key	E	L	H	High	Mitigated
F7	Public control-plane endpoint	I/D	M	M	Med	Fixed
F8	Sensitive content mirrored off-host	I	M	M	Med	Mitigated
F9	Capability gate bypassed by F1	E	M	M	Med	Fixed
F10	Deploy job runs out-of-tree script	T/E	L	H	Med	Fixed
F11	Website defacement via file-write	T	M	L	Low	Fixed
F12	Supply-chain (pinning)	T	L	M	Low	Follow-up
F13	Data-at-rest granularity	I	L	M	Low	Follow-up

* F5 is accepted in a single-operator context: its likelihood is controlled by hardware-backed 2FA and endpoint discipline, while its impact is inherent to consolidating an estate in one account. A multi-operator deployment distributes it. “Mitigated” denotes meaningful likelihood/impact reduction short of full elimination.

Appendix B ATT&CK technique coverage

The top attack paths were mapped to MITRE ATT&CK so the analysis aligns with defender tooling. Summary of the techniques considered and the control that addresses each.

T1059 — Command execution	Root job execution	No unattended path; human-gated confirm-runner
T1068 — Privilege escalation	The F1 chain	Category eliminated; staged→human→run only
T1552 — Unsecured credentials	F2 exfil path	Secret source removed from job allowlist; keyless auth
T1078 — Valid accounts	Leaked secret (TA2)	Bound identity + edge allowlist + rotation
T1190 — Exploit public-facing app	Public endpoint (F7)	Cloud Armor deny-by-default; direct-VM lock
T1195 — Supply-chain compromise	Dependencies (F12)	Version pinning; hash-pinning recommended
T1213 — Data from repositories	Off-host mirror (F8)	Sensitive content kept out of shared history
T1499 — Endpoint denial of service	Control-plane availability	Edge rate control; validated cutovers

Appendix C Methodology & references

- **STRIDE** — threat classification across Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege; applied per element.
- **NIST SP 800-30** — guide for conducting risk assessments; basis of the Likelihood × Impact register in Appendix A.
- **NIST SP 800-53** — security & privacy control families; basis of the mapping in §12 (as orientation, not certification).
- **MITRE ATT&CK** — adversary technique taxonomy; basis of Appendix B.
- **Attack trees** — goal-oriented AND/OR decomposition of the top objectives, used to expose finding chains.

End of document · Paracoding Security Architecture & Threat-Model Whitepaper · v1.0